ADAMANTIUM GAMING HUB

Software Engineering Project Report

War Thunder Mobile Community Hub

By TEAM AKAGAMI

SUBMISSION DETAILS	
Team Members	L Surya Uddish, S Rudrakshini, J Aryan
College / department	Data Science Department, Sreenidhi Institute of Science and Technology.
Course / subject	Web Technologies
Faculty guide	B. Vipul Anand Sir
Submission date	xx-10-2026

Contents

Abstract
1. Introduction
2. Software Requirements Specification (SRS)
3. System Analysis and Design
4. UML Diagrams
5. Implementation
6. Software Testing
7. Project Planning
8. Conclusion
9. References

Abstract 4

1. Introduction 4

1.1 Project overview 4

1.2 Objectives 4

1.3 Scope 4

2. Software Requirements Specification (SRS) 4

2.1 User classes 4

2.2 Functional requirements 5

2.3 Non-functional requirements 5

3. System Analysis and Design 5

3.1 Architecture 5

3.2 Data model and ER diagram 6

3.3 Use-case analysis 7

4. UML Diagrams 7

4.1 Use-case diagram 7

4.2 Class / domain model 7

4.3 Sequence diagram: request to join 8

4.4 Activity diagram: room creation 8

5. Implementation 9

5.1 Technologies 9

5.2 Application modules 9

5.3 Database and security design 9

6. Software Testing 9

7. Project Planning 10

7.1 SDLC model 10

7.2 Proposed schedule 10

7.3 Risk analysis 11

8. Conclusion 11

9. References 11

Abstract

Adamantium Gaming Hub is a college project that presents a War Thunder Mobile themed community gaming hub. It brings together a landing page, sign-in page, and three mode pages for ground vehicles, aircraft, and naval vessels. The mode pages combine vehicle catalogs and tactical guides with community rooms, join-request workflows, and room scoreboards. The front end is built with HTML, CSS, and JavaScript; Supabase provides hosted authentication and a PostgreSQL database accessed through the Supabase JavaScript client. Row Level Security (RLS) policies control database operations. This report describes the requirements, architecture, data model, implementation, testing plan, schedule, risks, results, limitations, and possible extensions.

1. Introduction

1.1 Project overview

Adamantium Gaming Hub is an independent fan-made web project created for a college assignment. It provides a single place to browse War Thunder Mobile themed ground, air, and naval content. The interface uses a dark visual theme with warm, muted champagne gold layout, image showcases, navigation between mode pages, and Supabase-backed community functions.

1.2 Objectives

- Present ground vehicles, aircraft, and naval vessels in separate browsable catalogs.
- Provide a shared sign-in flow for users who want to create rooms or participate in room workflows.
- Allow signed-in users to create and manage their own rooms and submit join requests to other eligible rooms.
- Show room scoreboards and mode-specific tactical guides.
- Keep access controlled through Supabase authentication and database RLS policies.

1.3 Scope

The project covers a static browser-based front end and a hosted Supabase backend. It includes the home page, login page, and ground, aviation, and naval pages. It **does not implement** live gameplay, matchmaking with War Thunder servers, payments, native mobile applications, or an official relationship with the game publisher. Showcase media is stored as local project assets or referenced by the page; it is separate from catalog records.

2. Software Requirements Specification (SRS)

2.1 User classes

User class	Typical capabilities
Visitor	Browse public catalog, guides, open rooms, and public scoreboards; sign in to use account features.
Signed-in player	Create a room, request to join another room, view own request state, and use account controls.
Room owner	Manage own room and review/respond to join requests, subject to the mode page and database policies.
Project maintainer	Manage catalog, guide, demo, and security data through the Supabase project.

2.2 Functional requirements

› The system shall provide navigation among Home, Tanks, Aviation, Naval, Rooms, Guides, and Login destinations where present.

› The system shall authenticate users through Supabase Auth and reflect the signed-in state in the interface.

› The system shall read vehicle, aircraft, vessel, and guide records from the database.

› The system shall list rooms by mode and allow an authenticated user to create a room.

› The system shall allow a user to request access to an eligible room and display the request status.

› The system shall allow room owners to respond to requests and manage rooms they own.

› The system shall display scores associated with a selected room and support refreshing the list.

› The system shall display clear loading, empty, and error states when data is unavailable.

2.3 Non-functional requirements

- Usability: consistent navigation, readable contrast, responsive layouts, and clear status feedback.
- Security: database access is protected by Supabase Auth and RLS; browser code must not contain service-role secrets.
- Reliability: data-loading failures should be reported without fabricating database records.
- Maintainability: mode pages and SQL scripts should remain organized and documented.
- Compatibility: pages should work in a modern browser with JavaScript enabled and network access to Supabase.
- Performance: keep local media reasonably sized and avoid unnecessary repeated database requests.

3. System Analysis and Design

3.1 Architecture

The browser loads static HTML, CSS, JavaScript, and media files. The Supabase JavaScript client communicates directly with the hosted Supabase API. Supabase Auth manages account sessions; the Data API applies PostgreSQL permissions and RLS policies before reading or changing public-schema rows. The current local workflow uses a development server. GitHub stores the project source; publishing the static site requires a hosting configuration and a deployed URL.

Figure 1. System context / data-flow overview

From	Relationship / action	To
Visitor or signed-in player	Uses pages, forms, catalog filters, room actions	Browser: index/login/tanks/aviation/ships
Browser pages	Loads HTML, CSS, JavaScript, and local assets	Local development server / static host
Browser JavaScript	Supabase client requests with public browser key and session token	Supabase Auth + Data API
Supabase Data API	Checks grants and RLS policies; reads/writes allowed rows	PostgreSQL public tables
PostgreSQL	Returns catalog, guide, room, request, and score data	Browser renders results

3.2 Data model and ER diagram

The current inventory discussed during database cleanup contains fourteen application tables: profiles, guides, three catalog tables, and three tables for each of the three room modes. Authentication identities are maintained by Supabase Auth in auth.users. Foreign keys connect room ownership and player/request identities to Auth users, while request and score rows reference their corresponding mode room. Some demo scoreboard entries are display-only and use a nullable player identity.

Figure 2. Entity relationships (simplified ER view)

From	Relationship / action	To
auth.users	1 → many; owner_id	tank_rooms / aviation_rooms / naval_rooms
auth.users	1 → many; requester_id	tank_room_requests / aviation_room_requests / naval_room_requests
tank_rooms	1 → many; room_id	tank_room_requests and tank_room_scores
aviation_rooms	1 → many; room_id	aviation_room_requests and aviation_room_scores
naval_rooms	1 → many; room_id	naval_room_requests and naval_room_scores
profiles	1 → 0/1 by matching id	auth.users
guides, vehicles, aircraft, naval_vessels	Catalog/content records; public read	Mode pages

Table group	Tables	Purpose
Identity	Profiles	Optional public profile information keyed to the Auth identity.
Content	Guides	Mode-specific guide text and author display name.
Catalogs	vehicles, aircraft, naval_vessels	Ground, air, and naval catalog entries.
Tank rooms	tank_rooms, tank_room_requests, tank_room_scores	Ground room lifecycle, join workflow, and scores.
Aviation rooms	aviation_rooms, aviation_room_requests, aviation_room_scores	Aviation room lifecycle, join workflow, and scores.
Naval rooms	naval_rooms, naval_room_requests, naval_room_scores	Naval room lifecycle, join workflow, and scores.

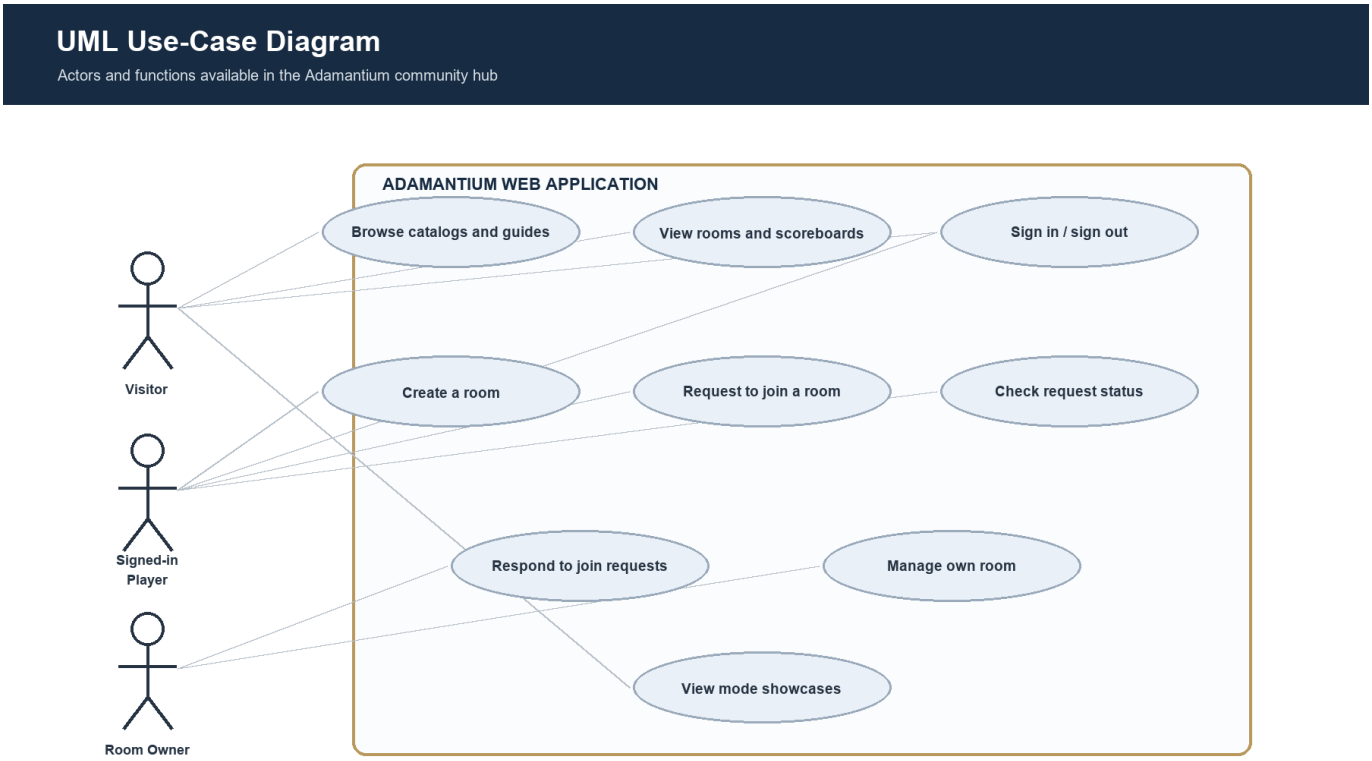
3.3 Use-case analysis

Actor	Use cases
Visitor	Browse catalogs and guides; inspect public/open rooms and public scoreboards; go to sign-in.
Signed-in player	Create room; send join request; check request status; view scoreboard; sign out.
Room owner	Review incoming requests; accept or reject; update/delete own real room where supported.
Maintainer	Seed or update catalog, guide, and demo records; maintain security policies.

4. UML Diagrams

The following UML diagrams model representative users, data entities, and room workflows in the current application. They are simplified models of the project's implementation.

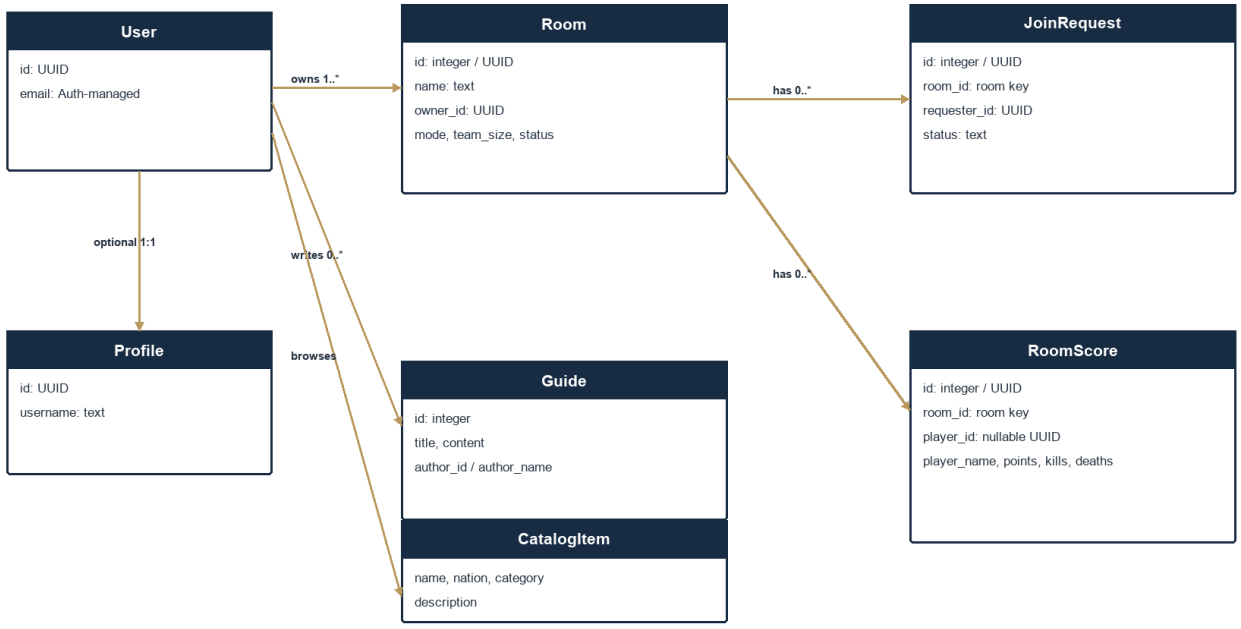
4.1 Use-case diagram



4.2 Class / domain model

UML Class Diagram

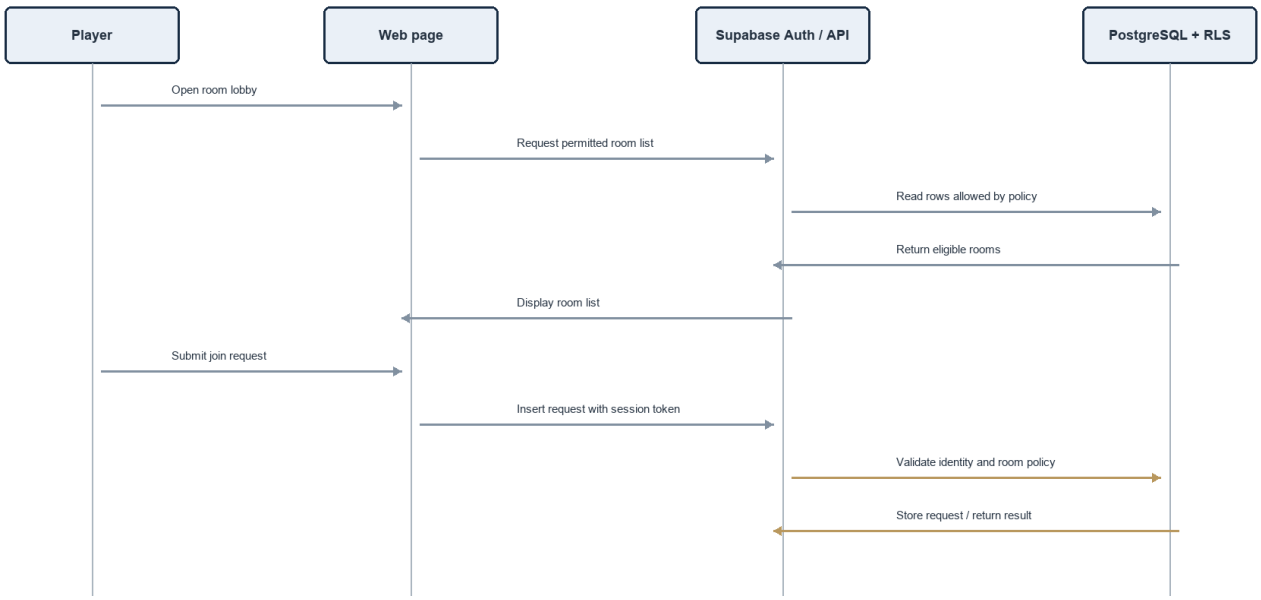
Conceptual data model for identity, catalogs, rooms, requests, and scores



4.3 Sequence diagram: request to join

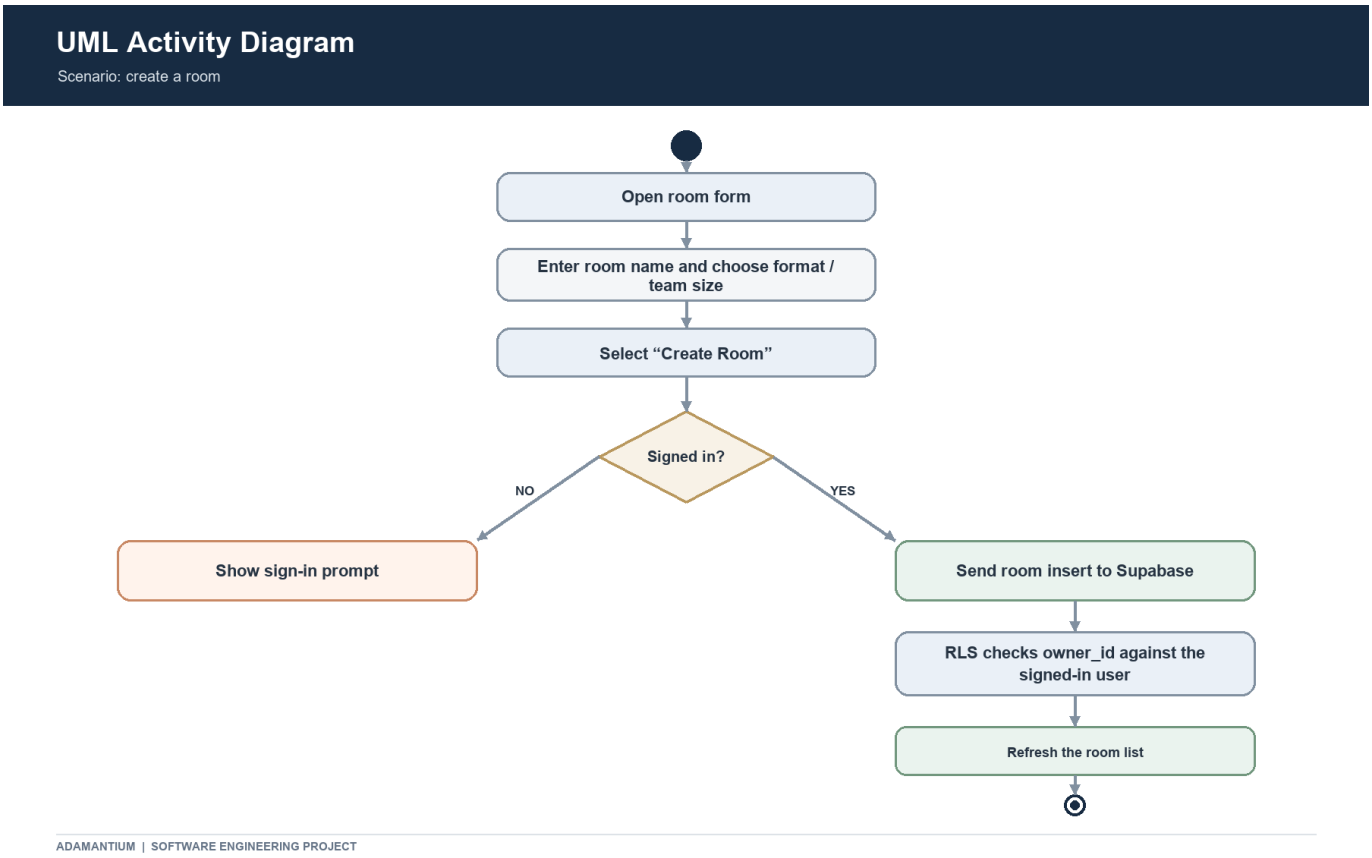
UML Sequence Diagram

Scenario: a signed-in player requests to join a room



After submission, the room owner reviews the pending request; the player sees the status on refresh.

4.4 Activity diagram: room creation



5. Implementation

5.1 Technologies

Layer	Technology / component	Use
Front end	HTML5, CSS3, vanilla JavaScript	Pages, design, interactions, data rendering.
Authentication and data access	Supabase JavaScript client v2 via CDN	Browser sign-in and PostgREST-style database operations.
Backend	Supabase Auth and PostgreSQL	Identity, relational records, access control.
Development	Visual Studio Code and local static server	Edit and preview the multi-page site.
Version control	Git and GitHub	Store and share the source repository.

5.2 Application modules

- Home module: introduces the hub and links to the mode pages.
- Authentication module: login.html uses Supabase Auth and returns users to the relevant page.
- Ground forces module: tanks.html loads vehicles/guides, rooms, requests, and scoreboards.
- Aviation module: aviation.html loads aircraft/guides, aviation rooms, requests, and scoreboards.
- Naval module: ships.html loads vessels/guides, naval rooms, requests, and scoreboards.

- Shared styling and scripts: style.css and script.js support visual consistency and common site behavior.

5.3 Database and security design

The current table inventory consists of profiles, guides, vehicles, aircraft, naval_vessels, and the room/request/score triplets for tanks, aviation, and naval. Saved SQL scripts are grouped under supabase/queries (including mode setup, guides, demo data, and security). The security script enables RLS and defines policies for public catalog/guide reads, room operations, request visibility and owner decisions, and score access as configured for each mode. The browser uses a publishable key; it is not a secret. A service-role key must never be placed in client-side HTML or JavaScript.

Repository note: the SQL currently lives as saved query scripts, while the project does not yet contain a confirmed ordered migration history pulled from the hosted database. Therefore, the scripts are useful project records but a clean, automated database rebuild has not been established by the repository alone.

6. Software Testing

The table below is a test plan. Do not submit invented actual results: run each case on the current local site and deployed site, then replace “To be recorded” with the observed result. The project owner has previously reported that the main features work locally, but this report does not treat that as a formal test log.

ID / scenario	Expected result	Actual result
T01 Open each page	Home, login, tanks, aviation, and ships load without broken layout or console errors.	Pass — all five pages opened in Chrome
T02 Browse catalogs and guides	Database entries render; empty/error states are clear when records are unavailable.	Pass — entries loaded on all three pages.
T03 Visitor attempts protected action	User is directed to sign in or sees an appropriate sign-in prompt.	Pass — sign-in was required.
T04 Sign in / sign out	Auth state updates and navigation returns to the intended page.	Pass — both actions worked.
T05 Create a room while signed in	Room is saved with current user as owner and appears after refresh.	Pass — room appeared in the list after refresh.
T06 Request to join a real room	Request is saved once and the requester sees its status.	Pass — request appeared as pending.
T07 Owner responds to request	Only the room owner can change the request status.	Pass — status changed and displayed correctly.
T08 Scoreboard display	Selected room scores load and refresh correctly, including demo display-only players.	Pass — scores loaded and refresh worked.
T09 Delete room	Owner can delete own eligible real room; demo room restrictions remain enforced.	Pass — my real room deleted; demo room had no delete option.
T10 Unauthorized database operation	RLS rejects requests that do not satisfy the table policy.	Pass — Supabase denied the unauthorized action.

7. Project Planning

7.1 SDLC model

An **iterative prototyping** approach best describes the project: build a page or feature, preview it locally, compare it with the intended design, connect the feature to Supabase, then refine based on observed behavior. This description is based on the project history and should be adjusted if the course requires a named formal SDLC model.

7.2 Proposed schedule

Phase	Work	Status / evidence
1. Requirements and visual plan	Define pages, mode content, and room actions.	Implemented pages exist; confirm dates.
2. Front-end build	Create shared navigation, page styling, showcases, and forms.	HTML/CSS/JS files are present.
3. Database integration	Connect catalogs, guides, rooms, requests, and scores; configure RLS.	Supabase scripts and page queries are present.
4. Data cleanup	Consolidate tables and remove unused social tables.	Current inventory provided.
5. Verification	Run local and hosted test cases and fix defects.	Fixed the defects of the test cases.
6. Publication	Configure static hosting and verify deployed paths and Supabase access.	Published.

7.3 Risk analysis

Risk	Impact	Mitigation
Incorrect RLS policy	Unauthorized read/write or blocked feature	Test anonymous, requester, owner, and other-user roles separately.
Schema drift between SQL scripts and hosted DB	Pages fail to load or write records	Keep ordered migrations and export/compare schema before changes.
Missing or incorrect asset paths after deployment	Images/video do not display	Use repository-relative paths and check case/spaces on deployed site.
Public client configuration misunderstood	Exposure of privileged credentials	Only publishable key in browser; never expose service-role key; retain RLS.
Large media files	Slow page loads or hosting limits	Compress media and verify repository/host limits before publishing.
Unverified test evidence	Report overstates completion	Record observed results, date, browser, and screenshots/logs.

8. Conclusion

Adamantium combines a multi-page War Thunder Mobile themed interface with Supabase-backed catalogs, guides, authentication, rooms, join requests, and scoreboards. The project demonstrates front-end development, relational data modeling, hosted authentication, and row-level access control. The main remaining report and release tasks are to record formal test outcomes, confirm the deployed hosting setup, verify all assets on the hosted site, and maintain a reproducible ordered database migration history. Potential future enhancements include a moderator/admin interface, stronger input validation and accessibility review, pagination/search for catalogs, and a documented automated deployment workflow.

9. References

- Supabase Documentation, “JavaScript Client Library” and “Authentication.” https://supabase.com/docs/reference/javascript/
- Supabase Documentation, “Row Level Security.” https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase Documentation, “Database Migrations.” https://supabase.com/docs/guides/deployment/database-migrations

- MDN Web Docs, HTML, CSS, and JavaScript references. https://developer.mozilla.org/
- War Thunder Mobile official website. Cite the exact page used for game descriptions or images: [https://wtmobile.com/en/].
- Adamantium project source files and Supabase SQL scripts, accessed.